Министерство образования Республики Беларусь
Учреждение образования
«Брестский государственный технический университет»
Факультет электронно-информационных систем
Кафедра интеллектуальных информационных технологий

Отчет по лабораторной работе №2

Специальность АС-68

Выполнил:
Студент 3-го курса
Группы АС-68
М.Р. Мартысюк
Проверил:
А. А. Крощенко
Ст. преп. кафедры ИИТ,
«___»_____2026 г.

Брест 2026

Цель: Изучить применение **линейной** и **логистической регрессии** для решения практических задач. Научиться обучать модели, оценивать их качество с помощью соответствующих метрик и интерпретировать результаты.

Вариант 6

- **Регрессия (Прогнозирование цен на жильё в Бостоне)**
  1. Boston Housing
  2. Предсказать медианную стоимость дома (MEDV)
  3. **Задания:**
     - загрузите данные. Используйте все признаки для обучения;
     - обучите модель **линейной регрессии**;
     - рассчитайте **MSE** и **R^2**;
     - визуализируйте зависимость MEDV от RM (среднее количество комнат) с линией регрессии.
- **Классификация (Диагностика рака груди)**
  1. Breast Cancer Wisconsin
  2. Предсказать, является ли опухоль злокачественной (malignant) или доброкачественной (benign)
  3. **Задания:**
     - загрузите данные и выполните стандартизацию признаков;
     - обучите модель **логистической регрессии**;
     - рассчитайте **Accuracy, Precision** и **Recall**;
     - постройте **матрицу ошибок** и объясните значение каждого из её квадрантов в контексте задачи.

Код программы 1:

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

# 1. Загрузка
df = pd.read_csv("BostonHousing.csv")
df = df.drop(columns=[c for c in ["Unnamed: 0", "CAT. MEDV"] if c in df.columns])
df = df.dropna()

# 2. Разделение 80/20
test = df.sample(frac=0.2, random_state=52)
train = df.drop(test.index)

# Формирование выборок
features = [c for c in df.columns if c != "MEDV"]
X_train = train[features].to_numpy(dtype=float)
X_test = test[features].to_numpy(dtype=float)
y_train = train["MEDV"].to_numpy(dtype=float)
y_test = test["MEDV"].to_numpy(dtype=float)

# 3. Линейная регрессия
```

```
add_bias = lambda X: np.column_stack([np.ones(len(X)), X])
beta, *_ = np.linalg.lstsq(add_bias(X_train), y_train, rcond=None)
y_pred = add_bias(X_test) @ beta

# 4. Метрики и вывод предсказаний
mse = np.mean((y_test - y_pred) ** 2)
r2 = 1 - np.sum((y_test - y_pred) ** 2) / np.sum((y_test - y_test.mean()) ** 2)
print(f"MSE: {mse:.3f}")
print(f"R2 : {r2:.3f}")

results = pd.DataFrame({
    'Реальная цена (тыс. $)': y_test,
    'Предсказанная цена': y_pred.round(2),
    'Разница': (y_test - y_pred).round(2)
})
print(results.sample(n=10, random_state=52))

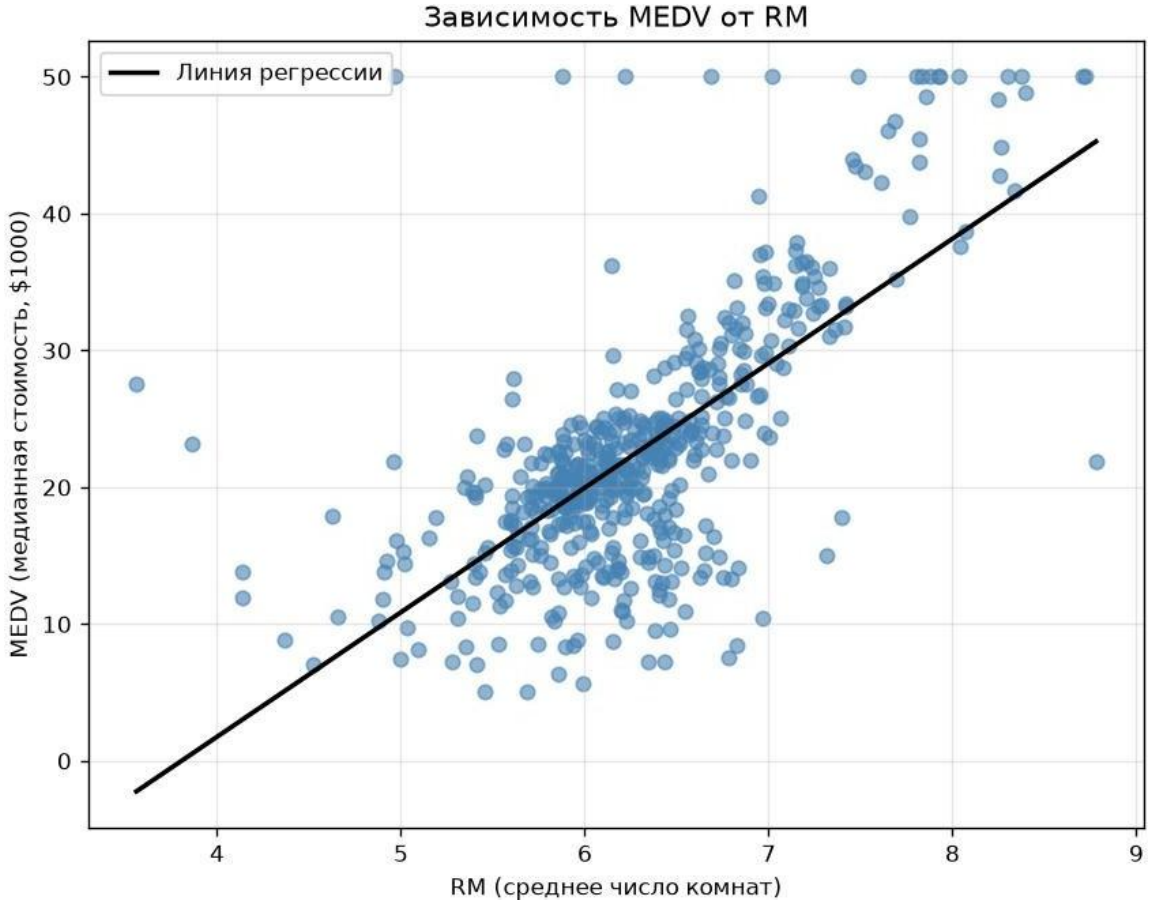
# 5. Визуализация (MEDV от RM)
k, b = np.polyfit(df["RM"], df["MEDV"], 1)
x_line = np.linspace(df["RM"].min(), df["RM"].max(), 100)

plt.figure(figsize=(8, 6))
plt.scatter(df["RM"], df["MEDV"], alpha=0.6, color="steelblue")
plt.plot(x_line, k * x_line + b, color="black", linewidth=2, label="Линия регрессии")
plt.xlabel("RM (среднее число комнат)")
plt.ylabel("MEDV (медианная стоимость, $1000)")
plt.title("Зависимость MEDV от RM")
plt.legend()
plt.grid(alpha=0.3)
plt.show()
```

Результат:

```
MSE: 22.716
R2 : 0.763
    Реальная цена (тыс. $)  Предсказанная цена  Разница
82                     9.5                15.23    -5.73
8                     16.3                12.36     3.94
44                    26.7                31.75    -5.05
19                    22.2                23.95    -1.75
2                     11.9                22.50   -10.60
73                    10.4                12.73    -2.33
37                    23.8                27.60    -3.80
89                    20.9                21.71    -0.81
67                    23.1                24.40    -1.30
18                    14.6                18.37    -3.77
```

Визуализация:



Код программы 2:

```
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, precision_score, recall_score,
ConfusionMatrixDisplay

# 1. Загрузка и очистка
df = pd.read_csv("breast_cancer.csv")
df = df.drop(columns=[c for c in df.columns if c.startswith("Unnamed") or c == "id"])

if "diagnosis" in df.columns:
    df = df.rename(columns={"diagnosis": "target"})
df["target"] = df["target"].astype(str).str.strip().map({"M": 0, "B": 1})
df = df.dropna()

X = df.drop(columns=["target"])
y = df["target"]

# 2. Разделение
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=52, stratify=y
)

# 3. Стандартизация и обучение
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

model = LogisticRegression(random_state=52)
model.fit(X_train_scaled, y_train)
y_pred = model.predict(X_test_scaled)
```

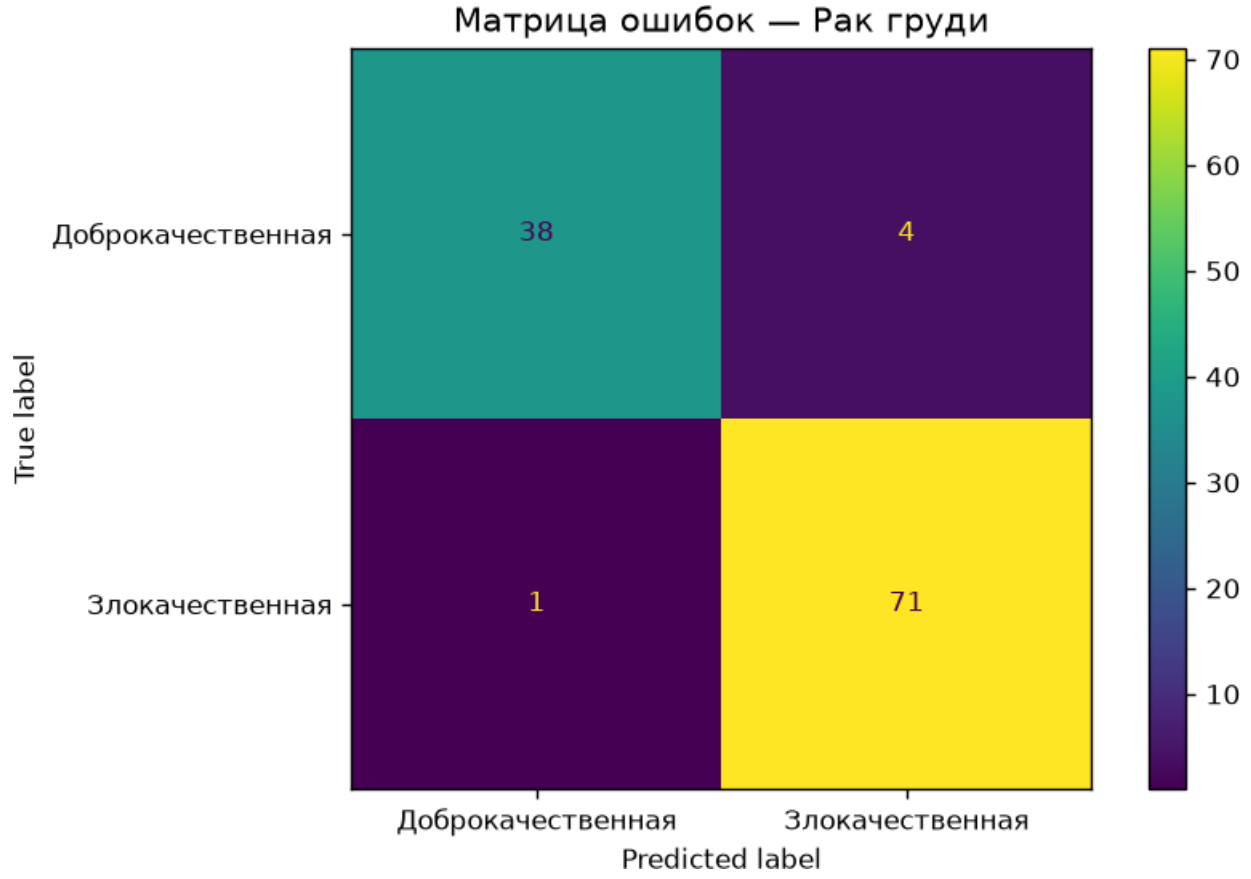
```
# 4. Вывод метрик
print(f"Общая точность: {accuracy_score(y_test, y_pred):.4f}")
print(f"Точность предсказания рака: {precision_score(y_test, y_pred,
pos_label=0):.4f}")
print(f"Полнота (сколько найдено больных раком): {recall_score(y_test, y_pred,
pos_label=0):.4f}")

# 5. Матрица ошибок
ConfusionMatrixDisplay.from_predictions(
    y_test, y_pred,
    display_labels=["Доброкачественная", "Злокачественная"]
)
plt.title("Матрица ошибок — Рак груди")
plt.show()
```

Результат:

```
Общая точность: 0.9561
Точность предсказания рака: 0.9744
Полнота (сколько найдено больных раком): 0.9048
```

Визуализация:



Вывод: изучено применение линейной и логистической регрессии для решения практических задач. Получены навыки по обучению моделей, оцениванию их качества с помощью соответствующих метрик и интерпретации результатов.